

Project 2 - Progress Summary

Architecture Decisions

- Created AnalysisDocument as a domain model independent of LangChain.
- Implemented PDFLoader using PyPDFLoader as an adapter to AnalysisDocument.
- Introduced DocumentAnalysisOrchestrator to coordinate document analysis services.
- Decided to keep common reusable components under src/common/.

LLM Infrastructure

- Implemented LLMService as an Abstract Base Class (ABC).
- Implemented GeminiLLMService and prepared the architecture for future providers (Ollama, OpenAI, Claude, Mistral).
- Implemented LLMConfiguration to represent LLM settings.
- Implemented LLMFactory using @staticmethod to create the configured LLM service.

Python Concepts Learned

- ABC (Abstract Base Class) defines interfaces/contracts for subclasses.
- @abstractmethod forces subclasses to implement required methods.
- @staticmethod is used for utility methods that do not require self or cls.
- @classmethod is commonly used for alternative constructors such as Configuration.load() or Document.from_pdf().
- __str__() provides human-readable representations.
- __repr__() provides developer/debug representations.
- Logging uses __str__ by default; %r in logger formatting invokes __repr__.
- Path objects from pathlib are preferable for file handling.

Configuration Design

- Configuration values will be stored in YAML files.
- A configs/ directory is recommended to separate configuration from source code.
- yaml.safe_load() converts YAML data into nested Python dictionaries.
- Configuration.load() will use @classmethod to create configuration objects directly from YAML.

Current Dependency Flow

- config.yaml -> Configuration.load() -> LLMConfiguration -> LLMFactory -> LLMService -> SummaryService -> DocumentAnalysisOrchestrator -> Streamlit UI

Next Steps

- Integrate YAML configuration loading.
- Implement SummaryService.

- Connect GeminiLLMService to the actual Gemini API through LangChain.
- Generate the first document summary from a PDF.
- Add a simple Streamlit UI for document upload and summarization.

The current architecture provides a reusable foundation shared between the RAG Playground and the AI Document Analyzer projects.